

**АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ
ПРОФЕССИОНАЛЬНАЯ ОБРАЗОВАТЕЛЬНАЯ ОРГАНИЗАЦИЯ
МОСКОВСКИЙ МЕЖДУНАРОДНЫЙ КОЛЛЕДЖ ЦИФРОВЫХ
ТЕХНОЛОГИЙ «АКАДЕМИЯ ТОП»**

**Юридический адрес: 121170, г. Москва, вн.тер.г. муниципальный округ
Дорогомилово, пр-кт Кутузовский, д.36, стр. 2, этаж I, помещение/ комната I/12,
тел.: +7(495)151-19-01,
ИНН 7730265193 КПП 773001001 ОГРН 1217700253594**

ОТЧЕТ

**о прохождении производственной практики по профессии
09.02.07 Информационные системы и программирование,
профессиональный модуль:**

Осуществление интеграции программных модулей

**Выполнил Баянов Алексей
Александрович**

Группа 9/3-РПО-23/1

Оценка _____

**Руководитель практики
Шульгин Александр Денисович**

Практика пройдена в

**Автономной некоммерческой организации профессиональной организации Московского
международного колледжа цифровых технологий «Академия ТОП» в городе Чита.**

Дата прохождения с «15» июня 2026 по «19» июня 2026

Москва, 2026

ЗАДАНИЕ

на производственную практику

Выдано студенту Баянову Алексею Александровичу группы 9/3-РПО-23/1

Автономной некоммерческой организации профессиональной организации

Московского международного колледжа цифровых технологий «Академия ТОП».

Программа подготовки 09.02.07 Информационные системы и программирование

Вид практики: производственная

Срок прохождения: с «15» июня 2026 по «19» июня 2026

Задание практики (в соответствии с программой практики):

- 1. Техническое задание**
- 2. Разработка модулей их интеграция и описание**
- 3. Описание разработки**
- 4. Тестирование кода**
- 5. Инспекция кода**
- 6. Выводы**
- 6. Инструкция по установке**

Руководитель производственной практики Шульгин Александр Денисович

Задание получил Баянов Алексей Александрович

Дата выдачи задания «15» июня 2026

Лист для замечания

Содержание

Введение.....	4
Практика 1. Создание БД в MySQL при помощи phpmyadmin.....	5
Практика 2. Создание клиентской части.....	13
Заключение.....	17

Введение

В данном проекте реализован интернет-магазин электроники. В рамках работы были выполнены следующие этапы: разработка технического задания (ТЗ), создание архитектуры и модулей системы, их описание, проведение тестирования кода, инспекция и анализ. Также подготовлены выводы по проделанной работе и инструкция по установке и запуску приложения.

Техническое задание

1.Обозначение системы: Интернет-магазин электроники.

2.Краткое описание системы: система предназначена для продажи электроники — телефонов, ноутбуков, планшетов и аксессуаров через онлайн-платформу. Пользователи могут искать товары, оформлять заказы, управлять личными данными.

1.1 Используемые технологии:

- Frontend: windows form;
- Backend: C#;
- БД: json;
- система контроля версий: Git;
- среда разработки: Visual studio.

1.2 Что продается электроника, включающая товары следующих категорий:

- телефоны;
- ноутбуки;
- планшеты;
- аксессуары (чехлы, зарядные устройства, наушники и др.).

1.3 Кто пользователи:

- покупатели: зарегистрированные и незарегистрированные пользователи, осуществляющие просмотр товаров, добавление их в корзину и оформление заказа;
- администратор: управляет каталогом товаров, просматривает заказы и редактирует данные системы.

1.4 Роли в системе:

- гость: просмотр каталога, добавление товаров в корзину (без оформления заказа);
- пользователь: зарегистрированный покупатель с доступом к личному кабинету и истории заказов;
- админ: полный доступ к управлению товарами и заказами.

2.1 Функциональные требования:

- каталог товаров: отображение списка товаров, поиск по названию, фильтрация по категориям, цене и популярности;
- корзина: возможность добавлять товары, изменять их количество, удалять товары;
- оформление заказа: форма с вводом данных покупателя, способами доставки и оплаты, сохранение информации о заказе;
- личный кабинет: просмотр истории заказов, изменение личных данных;
- административная панель: добавление, редактирование, удаление товаров; просмотр и управление заказами.

3.1 Нефункциональные требования:

- производительность: время отклика не более 1 секунды для операций поиска и фильтрации;
- надежность: обработка ошибок, логгирование ошибок, предотвращение сбоев;
- масштабируемость: возможность расширения каталога и возможностей системы без существенных изменений;
- работа на Windows: интерфейс реализован на Windows Forms с использованием C#.

4.1 Архитектура системы:

Основные компоненты:

- UI-модуль: форма Windows (Windows Forms), отвечающая за взаимодействие с пользователем;
- логика бизнес-процессов: слой обработки данных и команд, реализованный на C#;
- доступ к данным: взаимодействие с локальной или удаленной базой данных (например, SQLite или SQL Server).

Потоки данных:

- пользователь вводит данные через формы Windows Forms;
- данные передаются в бизнес-логический слой;
- обработка и сохранение в базе данных;
- вывод результата пользователю посредством формы.

Диаграмма компонентов (рисунок 1.1):

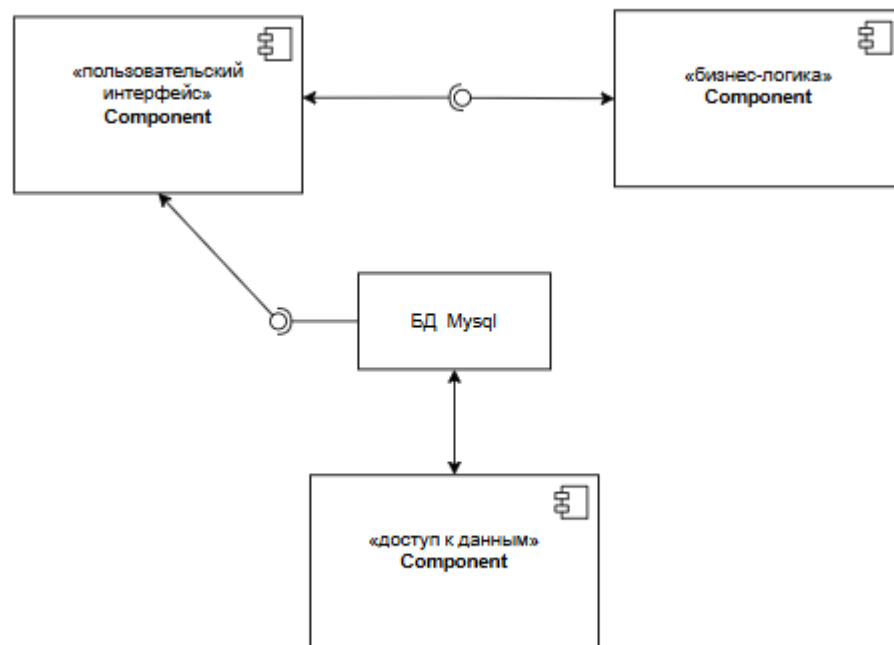


Рисунок 1.1 — Диаграмма компонентов.

Диаграмма вариантов использования (рисунок 1.2):

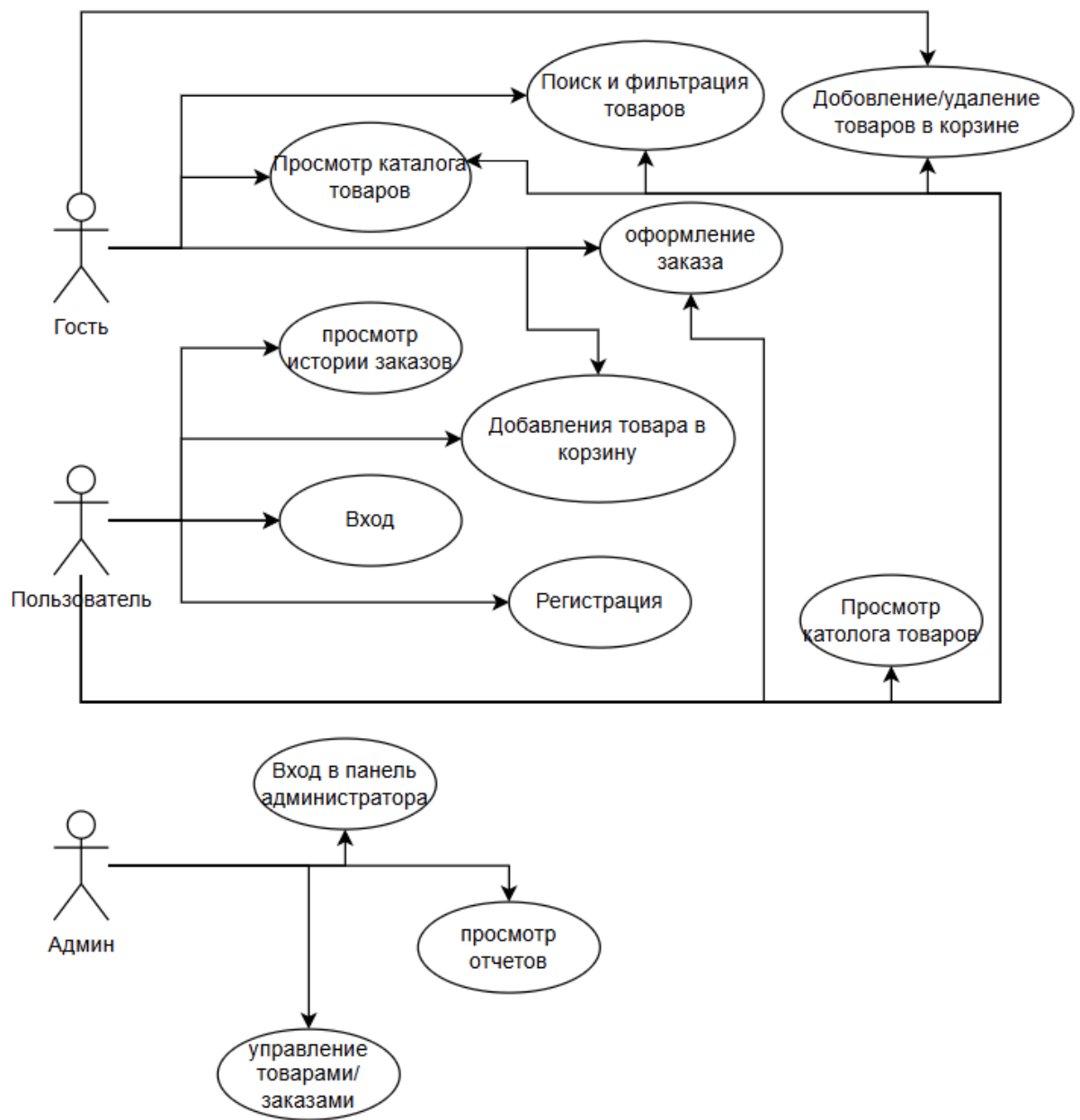


Рисунок 1.2 — Диаграмма вариантов использования.

Разработка модулей их интеграция и описание

Написание модулей и моделей на C#:

Модель 1. Каталог товаров (рисунок 2.1):

```
// Модель товара
Ссылка: 1
public class Product
{
    Ссылка: 0
    public int Id { get; set; }
    Ссылка: 0
    public string Name { get; set; }
    Ссылка: 0
    public string Category { get; set; }
    Ссылка: 1
    public decimal Price { get; set; }
    // Основные параметры для продукта
}
```

Рисунок 2.1 — Модель каталога товаров.

Модель 2. Корзина (рисунок 2.2):

```
// Модель корзины и элемента корзины
Ссылка: 1
public class CartItem
{
    Ссылка: 1
    public Product Product { get; set; }
    Ссылка: 1
    public int Quantity { get; set; }

    Ссылка: 0
    public decimal TotalPrice => Product.Price * Quantity;
}
```

Рисунок 2.2 — Модель корзина.

Модель 3. Оформление заказа (рисунок 2.3):

```
//Модель заказа
Ссылка: 0
public class Order
{
    Ссылка: 0
    public string OrderNumber { get; set; }
    Ссылка: 0
    public List<CartItem> Items { get; set; }
    Ссылка: 0
    public string CustomerName { get; set; }
    Ссылка: 0
    public string Address { get; set; }
    Ссылка: 0
    public string Phone { get; set; }
    Ссылка: 0
    public string Email { get; set; }
    Ссылка: 0
    public string Status { get; set; } // Новый, В обработке и т.д.
    Ссылка: 0
    public DateTime OrderDate { get; set; }
}
```

Рисунок 2.3 — Модель оформления заказа.

Модуль 1. Каталог товаров (рисунок 2.4):

```
// модуль каталог товара
Ссылка: 0
public class CatalogService
{
    private const string ProductsFilePath = "products.json";

    Ссылка: 0
    public List<Product> LoadProducts()
    {
        try
        {
            var json = File.ReadAllText(ProductsFilePath);
            var products = JsonConvert.DeserializeObject<List<Product>>(json);
            return products;
        }
        catch (Exception ex)
        {
            // логика логирования
            Console.WriteLine($"Ошибка загрузки товаров: {ex.Message}");
            return new List<Product>();
        }
        //Коммит: реализована загрузка и отображение товаров из JSON файла в каталоге.
    }
}
```

Рисунок 2.4 — Модуль каталога товаров.

Модуль 2. Корзина (рисунок 2.5 - 2.6):

```
// модуль корзина
Ссылка: 0
public class ShoppingCart
{
    Ссылка: 8
    public List<CartItem> Items { get; private set; } = new List<CartItem>();

    Ссылка: 0
    public void AddProduct(Product product)
    {
        var existingItem = Items.FirstOrDefault(i => i.Product.Id == product.Id);
        if (existingItem != null)
        {
            existingItem.Quantity++;
        }
        else
        {
            Items.Add(new CartItem { Product = product, Quantity = 1 });
        }
    }

    Ссылка: 0
    public void RemoveProduct(Product product)
    {
        var item = Items.FirstOrDefault(i => i.Product.Id == product.Id);
        if (item != null)
        {
            Items.Remove(item);
        }
    }

    Ссылка: 0
    public void ChangeQuantity(Product product, int quantity)
    {
        var item = Items.FirstOrDefault(i => i.Product.Id == product.Id);
        if (item != null)
        {
            item.Quantity = quantity;
        }
    }

    Ссылка: 0
    public decimal GetTotalSum()
    {
        return Items.Sum(i => i.TotalPrice);
    }

    Ссылка: 0
    public void SaveToFile()
    {
        try
        {
            var json = JsonConvert.SerializeObject(Items);
            File.WriteAllText("cart.json", json);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Ошибка сохранения корзины: {ex.Message}");
        }
    }
}
```

Рисунок 2.5 — Модуль корзина 1 часть.

```

Ссылка 0
public void LoadFromFile()
{
    try
    {
        if (File.Exists("cart.json"))
        {
            var json = File.ReadAllText("cart.json");
            Items = JsonConvert.DeserializeObject<List<CartItem>>(json) ?? new List<CartItem>();
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Ошибка загрузки корзины: {ex.Message}");
    }
    // реализована логика корзины: добавление, удаление товаров, изменение количества и подсчет общей суммы
}

```

Рисунок 2.6 — Модуль корзина 2 часть.

Модуль 3. Оформления заказа (рисунок 2.7):

```

// модуль оформление заказа
Ссылка 0
public class OrderService
{
    private const string OrdersFilePath = "orders.json";

    Ссылка 0
    public void SaveOrder(Order order)
    {
        try
        {
            List<Order> orders = new List<Order>();
            if (File.Exists(OrdersFilePath))
            {
                var json = File.ReadAllText(OrdersFilePath);
                orders = JsonConvert.DeserializeObject<List<Order>>(json) ?? new List<Order>();
            }

            // Генерируем уникальный номер заказа
            order.OrderNumber = Guid.NewGuid().ToString().Substring(0, 8).ToUpper();
            order.OrderDate = DateTime.Now;
            orders.Add(order);

            var updatedJson = JsonConvert.SerializeObject(orders);
            File.WriteAllText(OrdersFilePath, updatedJson);
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Ошибка сохранения заказа: {ex.Message}");
        }
    }

    Ссылка 0
    public string GenerateOrderNumber()
    {
        return Guid.NewGuid().ToString().Substring(0, 8).ToUpper();
    }
}
// добавлено оформление заказа, сбор данных, валидация и сохранение в JSON с уникальным номером

```

Рисунок 2.7 — Модуль оформление заказа.

Модуль 4. Административная панель (рисунок 2.8):

```
// Административная панель
Ссылка 0
public class AdminService
{
    private const string OrdersFilePath = "orders.json";
    private const string ProductsFilePath = "products.json";

    Ссылка 1
    public List<Order> GetAllOrders()
    {
        try
        {
            if (File.Exists(OrdersFilePath))
            {
                var json = File.ReadAllText(OrdersFilePath);
                return JsonConvert.DeserializeObject<List<Order>>(json) ?? new List<Order>();
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Ошибка получения заказов: {ex.Message}");
        }
        return new List<Order>();
    }

    Ссылка 0
    public void ChangeOrderStatus(string orderNumber, string newStatus)
    {
        try
        {
            var orders = GetAllOrders();
            var order = orders.FirstOrDefault(o => o.OrderNumber == orderNumber);
            if (order != null)
            {
                order.Status = newStatus;
                // сохранить
                var json = JsonConvert.SerializeObject(orders);
                File.WriteAllText(OrdersFilePath, json);
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Ошибка обновления статуса: {ex.Message}");
        }
    }
}

//внедрена обработка ошибок try-catch, логирование и интеграция модулей для корректной работы системы
// Методы для управления товарами аналогично
//реализована административная панель для просмотра заказов и изменения их статусов
```

Рисунок 2.8 — Модуль административная панель.

Описание модулей:

1. Модуль Каталог товаров (CatalogService)

Описание:

Этот модуль отвечает за хранение и загрузку информации о товарах, представленных в магазине. Он читает данные из JSON-файла (или другого источника), преобразует их в объекты Product и предоставляет их для отображения пользователю.

Задачи:

- загрузка перечня товаров из файла;
- передача списка товаров для отображения в интерфейсе;
- обеспечение возможности обновления каталога при

необходимости.

2. Модуль Корзина покупателя (ShoppingCart)

Описание:

Этот модуль управляет текущим списком выбранных товаров (корзиной). Он позволяет добавлять, удалять, изменять количество товаров и считать общую сумму заказа. Также реализована возможность сохранять состояние корзины между сессиями (например, через файл).

Задачи:

- добавление товара в корзину;
- удаление из корзины;
- изменение количества товара;
- подсчет итоговой суммы заказа;
- сохранение и загрузка корзины из файла (cart.json) для

сохранения состояния.

3. Модуль Оформление заказа (OrderService)

Описание:

Этот модуль занимается подготовкой и сохранением заказов. Он собирает данные клиента и товары из корзины, генерирует уникальный номер заказа, сохраняет историю заказов в JSON-файл.

Задачи:

- генерация уникального номера заказа;
- передача данных заказа на сохранение;
- сохранение всех заказов в отдельный файл (orders.json) для

дальнейшего просмотра.

4. Модуль Административная панель (AdminService)

Описание:

Этот модуль предназначен для управления заказами — просмотра, фильтрации, изменения статусов заказов и аналитики. Он работает с сохраненными данными по заказам, обеспечивая административный контроль.

Задачи:

- загрузка всех заказов из файла;
- обновление статусов заказов;
- обеспечение доступа к истории заказов и их управлению.

5. Модуль Интеграция и обработка ошибок

Описание:

Это общий слой, обеспечивающий взаимодействие всех вышеописанных модулей. Реализована обработка исключений (try-catch) для предотвращения сбоев, логирование ошибок и обеспечение стабильно работы системы. Также включает систему контроля версий Git с осмысленными коммитами.

Задачи:

- объединение модулей в целостную систему;
- передача данных между модулями (например, из каталога в корзину, из корзины в заказ);
- обработка ошибок с логированием для диагностики и устранения ошибок;
- версионный контроль с помощью Git.

Схема взаимодействия модулей (рисунок 2.9):

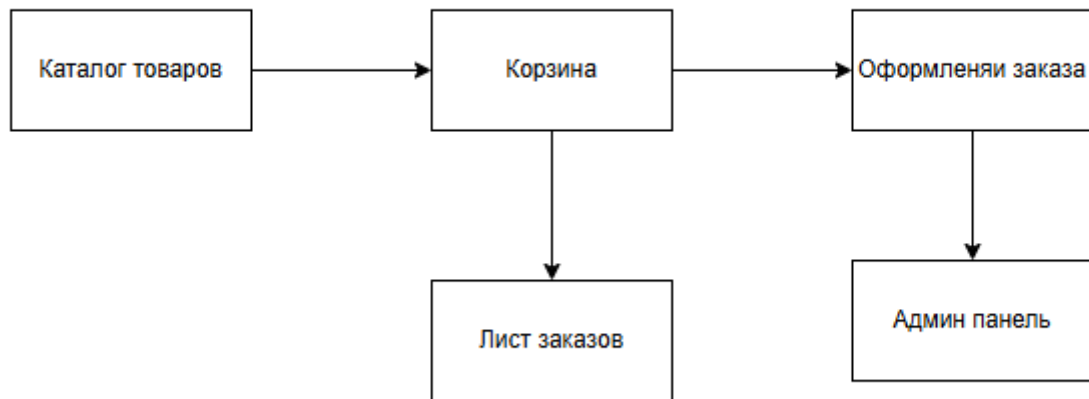


Рисунок 2.9 — Схема взаимодействия модулей.

Язык программирования: C# — основной язык для бизнес-логики и модели данных.

Среда разработки: Visual Studio — для написания и отладки кода.

Архитектура: MVVM (Model-View-ViewModel) — для организации интерфейса и взаимодействия с данными.

Фреймворк UI: WPF (Windows Presentation Foundation) — для создания десктопных XAML-интерфейсов.

XAML — разметка UI.

Работа с данными: Newtonsoft.Json (Json.NET) — для сериализации и десериализации JSON. Встроенные классы System.IO — для работы с файлами (чтение, запись).

Хранение данных: локальные JSON-файлы — для хранения товаров, корзины и заказов.

Описание разработки

Этапы разработки приложения:

Разработка выделенных модулей (рисунок 3.1):

```
// модуль каталог товара
Ссылка 0
public class CatalogService
{
    private const string ProductsFilePath = "products.json";

    Ссылка 0
    public List<Product> LoadProducts()
    {
        try
        {
            var json = File.ReadAllText(ProductsFilePath);
            var products = JsonConvert.DeserializeObject<List<Product>>(json);
            return products;
        }
        catch (Exception ex)
        {
            // логика логирования
            Console.WriteLine($"Ошибка загрузки товаров: {ex.Message}");
            return new List<Product>();
        }
    }
    //Коммит: реализована загрузка и отображение товаров из JSON файла в каталоге.
}
```

Рисунок 3.1 — Пример на модуле каталога товара.

Разработка интерфейса для интернет-магазина электроники (рисунок 3.2):

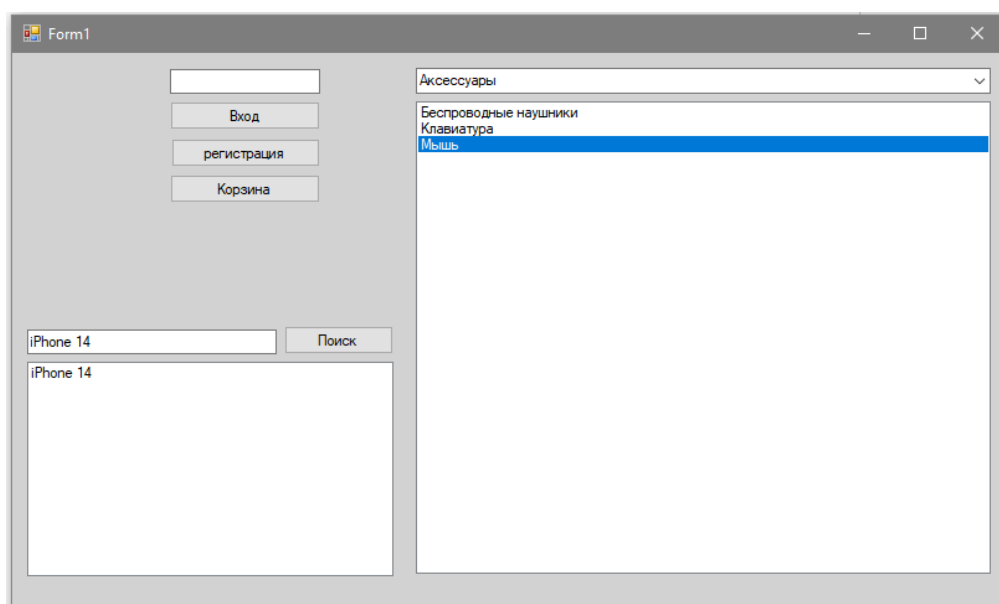


Рисунок 3.2 — Интерфейс приложения.

Пример кода приложения (рисунок 3.3):

```
// Метод поиска и отображения результатов
Ссылка 2
private void PerformSearch()
{
    string searchText = textBox1.Text.ToLower(); // приводим к нижнему регистру для удобства
    listBox2.Items.Clear();

    foreach (var product in allProducts)
    {
        if (product.ToLower().Contains(searchText))
        {
            listBox2.Items.Add(product);
        }
    }

    if (listBox2.Items.Count == 0)
    {
        listBox2.Items.Add("Нет результатов");
    }
}
```

Рисунок 3.3 — Пример кода метод поиска и отображение результатов.

Тестирование кода

Таблица 1.1 — Проведение тестирования приложения:

№	Тестируемый функционал	Описание проверки	Результат	Ошибки / Замечания	Предложенные исправления или комментарии
1	Загрузка списка категорий	Проверка, что категории отображаются при запуске формы	Успешно отображаются	Нет	Всё работает как нужно.
2	Выбор категории и отображение товаров	Проверка, что при выборе категории товары подсвечиваются	Успешно отображаются товары	Нет	Истинно. Большая часть логики реализована правильно.
3	Поле поиска (ввод текста)	Проверка фильтрации товаров по вводу в поисковый текст	Результаты фильтрации выводятся	Нет	Можно расширить тесты на точное совпадение, нечувствительность к регистру.
4	Кнопка "Добавить в корзину"	Проверка на добавление выделенного объекта в корзину.	Требуется доработка	Требуется доработка с выбором из списка категории товара	Рекомендуется реализовать кнопку и логику добавления товара в корзину.
5	Сохранение товаров в JSON	Проверка сохранения тестовых товаров в products.json	Успешно сохраняется	Нет	Можно добавить автоматическую проверку содержимого файла.
6	Загрузка товаров из JSON	Проверка загрузки товаров (через CatalogService) после сохранения файла	Успешно	Нет	Рекомендуется протестировать загрузку продуктов и отображение их в интерфейсе.

7	Регистрация заказа	проверка регистрации пользователя	Не,работает	требуется дальнейшая реализация данной функции	Также рекомендуется протестировать создание, сохранение и дальнейшее чтение заказов.
8	Обработка ошибок (например, файл не найден)	Искусственно отключите файл, чтобы проверить обработку ошибок	Обработка происходит успешно	Нет	В коде есть try-catch, ошибки выводятся в консоль, все работает правильно.
9	Изменение статуса заказа (админка)	Не тестировалась полностью.	Не, работает	Требуется реализовать вызовы ChangeOrderStatus(), протестируйте смену статуса	Полезно напечатать список заказов и изменить их статусы через интерфейс для проверки.
10	Вывод результатов в интерфейс	Все результаты отображаются корректно	Да, работает	Нет	В целом, UI части работают, можно расширять.

Таблица 1.1 — Проведение тестирования.

Инспекция кода

Вследствие выполненных тестов были выявлены некоторые ошибки которые следует решить (рисунок 5.1):

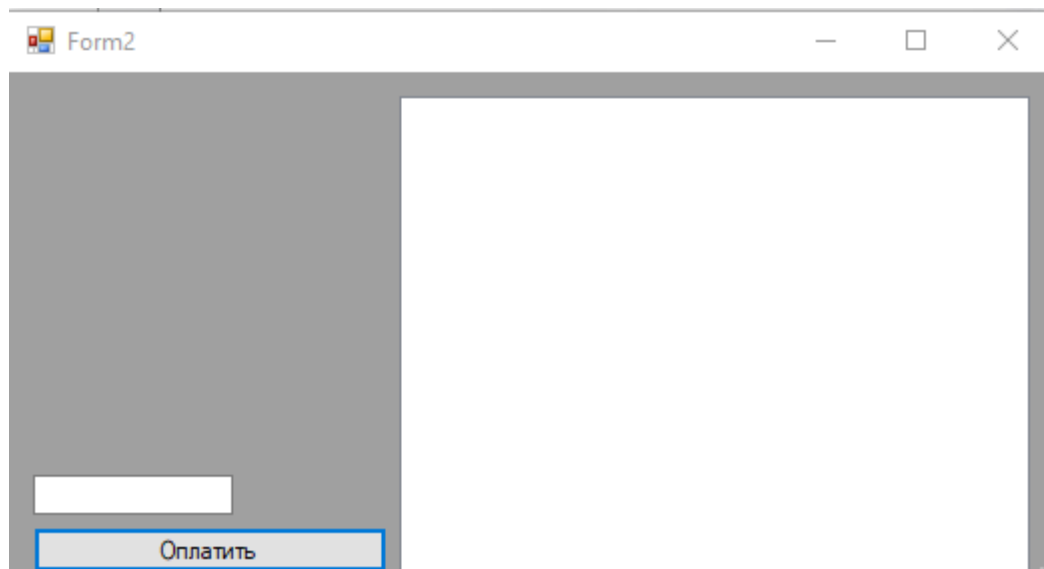


Рисунок 5.1 — Доделанная корзина.

Проверка регистрации пользователя (рисунок 5.2 - 5.3):

```
Ссылка 0
private void buttonRegister_Click(object sender, EventArgs e)
{
    string login = textBoxLogin.Text.Trim();
    string password = textBoxPassword.Text.Trim();
    string email = textBoxEmail.Text.Trim();

    // Проверка, что поля не пустые
    if (string.IsNullOrEmpty(login) || string.IsNullOrEmpty(password) || string.IsNullOrEmpty(email))
    {
        MessageBox.Show("Пожалуйста, заполните все поля.");
        return;
    }

    // Дополнительные проверки, например, формат email
    if (!IsValidEmail(email))
    {
        MessageBox.Show("Пожалуйста, введите корректный email.");
        return;
    }

    // Проверка уникальности логина
    if (IsLoginExist(login))
    {
        MessageBox.Show("Этот логин уже зарегистрирован. Выберите другой.");
        return;
    }

    // Регистрация (добавление пользователя в базу или список)
    RegisterUser(login, password, email);

    MessageBox.Show("Регистрация успешно завершена!");
}

// Метод проверки корректности email
Ссылка 1
private bool IsValidEmail(string email)
{
    try
    {
        var addr = new System.Net.Mail.MailAddress(email);
        return addr.Address == email;
    }
    catch
    {
        return false;
    }
}
```

Рисунок 5.2 — Доделанная проверка пользователя часть 1.

```

// Метод проверки существования логина
Ссылка: 1
private bool IsLoginExist(string login)
{
    // Например, проверка в списке зарегистрированных пользователей
    // В реальности – база данных или файл
    return users.Any(u => u.Login.Equals(login, StringComparison.OrdinalIgnoreCase));
}

// Метод регистрации пользователя
Ссылка: 1
private void RegisterUser(string login, string password, string email)
{
    // Добавьте нового пользователя в список или сохрани на диск
    users.Add(new User { Login = login, Password = password, Email = email });
}

// Класс для пользователя
Ссылка: 3
public class User
{
    Ссылка: 2
    public string Login { get; set; }
    Ссылка: 1
    public string Password { get; set; }
    Ссылка: 1
    public string Email { get; set; }
}

// Коллекция зарегистрированных пользователей
private List<User> users = new List<User>();

```

Рисунок 5.3 — Доделанная проверка пользователя часть 2.

Выводы

За время работы над проектом удалось реализовать основные модули интернет-магазина, включая загрузку товаров из JSON-файла, работу с корзиной и оформление заказов. В процессе были созданы модели данных для товаров, заказов и корзины. Также реализована простая проверка регистрации пользователей и сохранение данных.

Трудности и сложности:

- обработка ошибок при чтении и записи файлов;
- связь между моделями и их отображение в интерфейсе;
- реализация проверки уникальности логинов и правильности данных;
- некоторые функции, например, оформление заказа, требуют доработки.

Применённые навыки:

- работа с файлами и JSON сериализацией/десериализацией;
- использование событий и обработчиков в WinForms;
- создание моделей данных и организация бизнес-логики;
- основы работы с исключениями и обработка ошибок.

Инструкция по установке и запуску

Установка Visual Studio:

- зайдите на сайт visualstudio.microsoft.com и скачайте бесплатную версию Community;
- установите Visual Studio с компонентами для разработки Windows Forms.

Скачивание и открытие проекта:

- распакуйте архив и откройте файл решения (.sln) в Visual Studio.

Настройка среды:

- проверьте наличие файла `products.json` в каталоге проекта — он должен содержать список товаров в формате JSON;
- можно создать файл вручную или сгенерировать через интерфейс.

Запуск проекта:

- нажмите кнопку "Start" или клавишу F5 для запуска программы в режиме отладки.

Работа с приложением:

- в интерфейсе выберите товары, добавляйте их в корзину, оформляйте заказ;
- для тестирования зарегистрируйте нового пользователя через форму регистрации (при необходимости доработайте интерфейс).

Заключение

В ходе работы над интернет-магазином электроники были успешно реализованы основные модули системы: загрузка и отображение товаров, корзина покупок, оформление заказов и регистрация пользователей. Реализация дала практический опыт работы с файлами, JSON-сериализацией, обработкой ошибок и созданием интерфейса на WinForms.

На практике возникли некоторые сложности при обработке ошибок и реализации бизнес-логики, которые были успешно преодолены путём доработки кода и организации структурированных методов. Благодаря выполненной работе были закреплены навыки программирования на C#, проектирования интерфейсов и организации данных.